Hantro VC9000D Series Video Decoder

FPGA Verification Guide

Document Revision 1.02
29 December 2023

This document is compatible with:
VC9000D series

Legal Notices

COPYRIGHT INFORMATION

This document contains proprietary information of VeriSilicon Holdings Co., Ltd. VeriSilicon reserves the right to make changes to any products herein at any time without notice. VeriSilicon does not assume any responsibility or liability arising out of the application or use of any product described herein, except as expressly agreed to in writing by VeriSilicon; nor does the purchase or use of a product from VeriSilicon convey a license under any patent rights, copyrights, trademark rights, or any other of the intellectual property rights of VeriSilicon or third parties.

DISCLOSURE/RE-DISTRIBUTION LIMITATIONS

The information contained herein is not to be used by or disclosed to third parties without the express written permission of an officer of VeriSilicon Holdings Co., Ltd.

(VeriSilicon Distribution Level **D: CONFIDENTIAL – NOT FOR REDISTRIBUTION**).

TRADEMARK ACKNOWLEDGMENT

VeriSilicon, the VeriSilicon logo, Hantro and the Hantro logo are the trademarks of VeriSilicon Holdings Co., Ltd. in the United States and/or other jurisdictions. All other trademarks are the property of their respective holders.

For our current distributors, sales offices, design resource centers, and product information, visit our web site located at http://www.verisilicon.com.

Preface

Acronyms

1080p	High Definition resolution of 1920 x 1080 progressive video
720p	High Definition resolution of 1280 x 720 progressive video
AHB	**A**dvanced **H**igh-performance **B**us
APB	**A**dvanced **P**eripheral **B**us
API	**A**pplication **P**rogramming **I**nterface
AVC	**A**dvanced **V**ideo **C**oding (aka H.264, MVC)
AVS	**A**udio **V**ideo Coding **S**tandard Workgroup of China, compression standard for audio and video
AXI	**A**dvanced e**X**tensible **I**nterface
bps	**B**its **P**er **S**econd
CIF	**C**ommon **I**nterchange **F**ormat (352 x 288 pixels)
CODEC	**Co**mpressor/**DEC**ompressor
DivX	**Di**gital **V**ideo E**X**press
fps	**F**rames **P**er **S**econd
GPU	**G**raphics **P**rocessing **U**nit
HD	**H**igh **D**efinition (HD=1980 pixels x1080 lines, progressive scan)
HEVC	**H**igh **E**fficiency **V**ideo **C**oding, a video coding standard **(JCT-VC)** (aka H.265)
IVF	A simple file format that transports raw VP8 data
JFIF	**J**PEG **F**ile **I**nterchange **F**ormat
JPEG	**J**oint **P**hotographic **E**xperts **G**roup, a lossy compression method for digital images
mbps	**M**ega (1000000) **B**its **P**er **S**econd
MC	**M**otion **C**ompensation
MPEG	**M**oving **P**ictures **E**xperts **G**roup, lossy compression standards for video and audio
MV	**M**otion **V**ector
MVC	**M**ultiview **V**ideo **Co**ding
PAL	**P**hase **A**lternate **L**ine (resolution 720 x 576 pixels)
RGB	**R**ed, **G**reen, **B**lue color space representation
RV	**R**eal **V**ideo
VC-1	SMPTE 421M **V**ideo **C**oding format (initially Windows Media Video 9)
VCCORE	**V**ideo **C**odec **C**ore
VPU	**V**ideo **P**rocessing **U**nit
WebM	A digital multimedia container file format promoted by the open-source WebM Project.
WebP	Lossy and lossless codec method for digital images, developed by Google
YCbCr	Color space with one luma and two chroma components used for digital decoding
YUV	Color format with luma and two chroma components

`Consolas` font is frequently used when describing code or parameters in the document. *Italic* expression is used in case of file names, paths, or commands, and in notes in text requiring special attention.

For additional HEVC acronyms, see http://www.hevcbook.de/acronyms/.

Table of Contents

1 Overview

This guide provides an overview of the flow for VC9000D FPGA testing on either an ARM platform or on a PC platform with a PCIE Interface. Some steps are the same for the two different platforms.

For clarity, the directories listed below are frequently referenced in the document.

Directory Reference	Description
CODE_BASE	Path of VC9000D release package
SOFTWARE_BASE	**CODE_BASE/software**
CACHE_BASE	**CODE_BASE/cache**
TESTBENCH_BASE	**CODE_BASE/software/test**
TEST_BINARY	The executable binary generated by software (which usually resides in **CODE_BASE/out/** subdirectories).

1.1 Requirements

Before proceeding, be sure you have the following available:

- VC9000D release packages (including FPGA bit files and Device Drivers)
- A working FPGA environment with drivers installed
- C-Model Binary
- VC9000D TestBench user manual reference document

1.2 References

The following VeriSilicon Hantro reference documents may be useful during FPGA testing.

- *Hantro VC9000D Video Decoder TestBench User Manual* describes the command line syntax for use with C-Model and the test binaries.
- *Hantro VC9000D Series Video Decoder Hardware Integration Guide* is the primary integration reference manual for this IP.
- *Hantro VC9000D Series Video Decoder Simulation Guide* describes the simulation environment including RTL test bench and simulation scripts for the IP core.

2 Build Driver and Test Binary

2.1 Build memalloc Driver

a. Navigate to the driver source code directory.
   Source code for the ARM platform is located in SOFTWARE_BASE/linux/memalloc.
   For the PCIE platform, source code is located in SOFTWARE_BASE/linux/memalloc_pc.

b. For the ARM platform, the user must make the following modifications:
   i. Change the KDIR to the Kernel directory in SOFTWARE_BASE/linux/memalloc/Makefile.
   ii. Add the tool chain directory into the system path.
   iii. Specify the CPU architecture and cross compiler.
   E.g., <make all ARCH=arm64 CROSS_COMPILE= aarch64-linux-gnu->
   iv. Reserve enough memory in the uboot start up stage for VC9000D and modify the following parameters in memalloc.c.
   - alloc_size // Size of memory which is reserved for VC9000D (in MB).
   - alloc_base // Start bus address of memory which is reserved for VC9000D.

   There are a total of three types of buffers that are allocated for decoding: the output frame buffer used to store frame data, the input stream buffer used to store stream and the working buffer for SW/HW interactive. A typical alloc_size should take these buffers into consideration. Here is an example on how to compute alloc_size:
   - **Frame buffer:** At least 7 buffers should be allocated. Here, 5 buffers for reference, 1 buffer for decoding and 1 buffer for output. On the other hand, 10-bit output and 4K resolution should also be taken into consideration. As a result, at least 120MB should be allocated. If the Post-processor feature is enabled, the frame buffer size should be doubled to 240MB.
   - **Stream buffer:** At least 1 buffer should be allocated to store one frame's bit stream. 10MB's stream buffer can handle most of bit stream rate.
   - **Working buffer:** A typical working buffer size is 8MB.
   - **Reserved buffer:** It is suggested to reserve 2 output buffers memory to avoid memory fragmentation. A typical reserved memory size is 35MB.

   If the PP feature is disabled, alloc_size should be > 173MB;
   if the PP feature is enabled, alloc_size should be > 293MB.

   For the PCIE platform, the user should modify the following parameters in memalloc.c.
   - PCI_DEVICE_ID_HANTRO_PCI // The PCIE ID
   - alloc_size // Size of memory which is reserved for VC9000D (in MB).
   - alloc_base // Start bus address of memory which is reserved for VC9000D.
     - In function PcieInit, alloc_base is assigned as "gBaseHdwr + offset". Here, "gBaseHdwr" is the PCI base address obtained via the macro "pci_resource_start" with IO BAR index 0 or another index depending on the PCIE device design. The "offset" indicates the relative address of the Hantro IP in the PCIE board. The user should modify this "offset" to the correct value.

   If the PP feature is disabled, alloc_size should be > 173MB;
   if the PP feature is enabled, alloc_size should be > 293MB.

c. Build the source code with command <make clean > <make all>. After a successful build, a driver named "memalloc.ko" is generated in the build directory.

2.2 Build VC9000D Register Access Device Driver

Step 1. Navigate to the driver source code directory.

Platform	Driver Source Code Directory
ARM	SOFTWARE_BASE/linux/subsys_driver
PCIE	SOFTWARE_BASE/linux/subsys_driver

Step 2. For the ARM platform, the user must make the following modifications:

i. Change the KDIR to the Kernel directory in SOFTWARE_BASE/linux/subsys_driver/Makefile if the default value does not work.

ii. Add the ARM Linux cross tool chain directory into the system path.

iii. Specify the CPU architecture and cross compiler.
E.g., <make all ARCH=arm64 CROSS_COMPILE= aarch64-linux-gnu->

Step 3. Edit subsys.c: modify the following parameters that list all the sub-systems and hardware cores in each sub-system.

i. subsys_array: a list of sub-systems, each of which is described as a tuple of <slice index, index, base address>.

```
// user needs to configure index and offset for each subsystem.
struct SubsysDesc subsys_array
```

SubsysDesc member	Description
slice_index	Always 0 and is reserved for future extension.
index	An indicator of a sub-system in a slice and differs with other sub-system with the same slice index.
base	For the ARM platform, the base address indicates the bus address of the sub-system. For the PCIE platform, the base address indicates the relative offset of the sub-system to the base of the PCIE board.

ii. core_array: a list of all the cores from all the sub-systems that are supported. Each core is described as a tuple of <slice id, subsys id, core type, offset, iosize, irq>

```
// Physical address offset for each Core's register map
struct CoreDesc core_array[]
```

CoreDesc member	Description
slice_id	Slice ID of the sub-system that this IP core is in.
subsys_id	Sub-system ID that this IP core is in.
core_type	The type of this IP core.
offset	The relative address of the core to the base address of the sub-system of the core.
iosize	Size (in bytes) of the register map for the IP core.
irq	IRQ number and should be set to (-1) if no IRQ is defined for the IP core.

Below is a simple example of a single sub-system of VC9000D with L2Cache and DEC400:

```
struct SubsysDesc subsys_array[] = {
  /* {slice_index, index, base} */
  {0, 0, 0x600000}
};
struct CoreDesc core_array[] = {
  /* {slice_id, subsys_id, core_type, offset, iosize, irq} */
  {0, 0, HW_VC9000D, 0x0,     503*4,  -1, 1},
  {0, 0, HW_L2CACHE, 0x40000, 231*4,  -1, 0},
  {0, 0, HW_DEC400,  0x80000, 1568*4, -1, 0}
```

```
};
```

Step 4. Build the source code with command `<make clean > <make all>.` After a successful build, a driver named "`hantrodec.ko`" is generated in the current directory.

3 Build VC9000D Test Binary

3.1 HEVC

Step 1. Navigate to the source code directory CODE_BASE.

Step 2. For an ARM platform build:

a. You may need to edit CODE_BASE/source/common/common.mk and make changes to adjust for your ARM cross compiler and ARM arch options. A quick way to set up these options is by using export.
For example:

```
export TOOLCHAIN=/home/vivscm/working/gcc-linaro-aarch64-linux-gnu-4.9-2014.07_linux
export PATH=$TOOLCHAIN/bin:$PATH
export CROSS=aarch64-linux-gnu-
export ARCH=-march=armv8-a
```

b. Build the binary with command <make clean g2dec ENV=arm_linux>

c. Because HEVC supports multi-core decoding, the user can build the binary with command <make clean g2dec ENV=arm_linux USE_MULTI_CORE=y>.

d. After building, the final TEST_BINARY will be generated as CODE_BASE/out/arm_linux/debug/g2dec. If compiling with the RELEASE=y option, debug in the above directory is changed to release.

For a PC PCIE build:

a. Build the binary with command <make clean g2dec ENV=x86_linux_pci >

b. After building, the final TEST_BINARY will be generated as CODE_BASE/out/x86_linux_pci/debug/g2dec. If compiling with the RELEASE=y option, debug in the above directory is changed to release.

3.2 H.264

Step 1. Navigate to the source code directory CODE_BASE.

Step 2. For an ARM platform build:

a. You may need to edit CODE_BASE/source/common/common.mk and make changes to adjust for your ARM cross compiler and ARM arch options. Similar to HEVC, a quick way to set up these options is by using export.

b. Build the binary with command <make clean g2dec ENV=arm_linux>

c. Because H.264 supports multi-core decoding and it uses libav to get the frame length, the user must compile libav with the cross compiler first, then build the binary with command <make clean g2dec ENV=arm_linux USE_MULTI_CORE=y LIBAV=*path_to_libav*>.

d. After building, the final TEST_BINARY will be generated as CODE_BASE/ CODE_BASE/out/arm_linux/debug/g2dec.

For a PC PCIE build:

a. Build the binary with command <make clean g2dec ENV=x86_linux_pci>

b. Because H.264 supports multi-core decoding and it uses libav to get the frame length, the user must compile libav with the cross compiler first, then build the binary with command <make clean g2dec ENV=x86_linux_pci USE_MULTI_CORE=y LIBAV=*path_to_libav*>.

c. After building, the final TEST_BINARY will be generated as: CODE_BASE/out/ x86_linux_pci/debug/g2dec.

3.3 JPEG

Step 1. Navigate to the source code directory CODE_BASE.

Step 2. For an ARM platform build:

a. You may need to edit CODE_BASE/source/common/common.mk and make changes to adjust for your ARM cross compiler and ARM arch options. Similar to HEVC, a quick way to set up these options is by using export.

b. Build the binary with command <make clean jpegdec ENV=arm_linux>

c. After building, the final TEST_BINARY will be generated as: CODE_BASE/ CODE_BASE/out/arm_linux/debug/jpegdec.

For a PC PCIE build:

a. Build the binary with command <make clean jpegdec ENV=x86_linux_pci>

b. After building, the final TEST_BINARY will be generated as CODE_BASE/out/x86_linux_pci/debug/jpegdec.

3.4 VP9

Step 1. Navigate to the source code directory CODE_BASE.

Step 2. For an ARM platform build:

a. You may need to edit CODE_BASE/source/common/common.mk and make changes to adjust for your ARM cross compiler and ARM arch options. Similar to HEVC, a quick way to set up these options is by using export.

b. Build the binary with command <make clean g2dec ENV=arm_linux>
The user must compile nestegg with the cross compiler first, then build the binary with command

`<make clean g2dec ENV=arm_linux WEBM_ENABLED=y NESTEGG=path_to_nestegg>`.

c. After building, the final TEST_BINARY will be generated as:
   CODE_BASE/ CODE_BASE/out/arm_linux/debug/g2dec.

For a PC PCIE build:

a. Build the binary with command `<make clean g2dec ENV=x86_linux_pci>`
b. The user must compile nestegg with the cross compiler first, then build the binary with command
   `<make clean g2dec ENV=x86_linux_pci WEBM_ENABLED=y NESTEGG=path_to_nestegg>`.
c. After building, the final TEST_BINARY will be generated as:
   CODE_BASE/out/x86_linux_pci/debug/g2dec.

4 Install Drivers and Run Test Binary

For the ARM platform, because the driver and binary are built in the host, the user should copy these files from host to the ARM platform. A suggested way is to use NFS to mount the working directory from host to the ARM platform.

4.1 Install memalloc Driver

Install the VC9000D memory allocation driver by running shell script `memalloc_load.sh`.

Script locations:

Platform	Script Location
ARM	SOFTWARE_BASE/linux/memalloc/memalloc_load.sh
PC PCIE	SOFTWARE_BASE/linux/memalloc_pc/memalloc_load.sh
PC PCIE +VCMD	SOFTWARE_BASE/linux/memalloc_pc/memalloc_load.sh vcmd_size=0x900000

Some logs will be printed to show whether the installation is successful.

Note: memalloc for ARM platform should be set to an uncacheable memory:

alloc_base // Start bus address of memory which is reserved for VC9000D.

// Make sure this memory region is uncacheable.

4.2 Install VC9000D Device Driver

Install the VC9000D Device Driver by running shell script `driver_load.sh`.

Script locations:

Platform	Script Location
ARM	SOFTWARE_BASE/linux/subsys_driver/driver_load.sh pcie=0
PC PCIE	SOFTWARE_BASE/linux/subsys_driver/driver_load.sh

Some logs will be printed to show whether the installation is successful or not.

4.3 Run Test Binary

After building the test binaries, the following binaries should be available in the following subdirectories.

Standard	TEST_BINARY Paths and Filenames
HEVC	ARM: **CODE_BASE/out/arm_linux/debug/g2dec** PC PCIE: **CODE_BASE/out/x86_linux_pci/debug/g2dec**
H.264	ARM: **CODE_BASE/out/arm_linux/debug/g2dec** PC PCIE: **CODE_BASE/out/x86_linux_pci/debug/g2dec**
JPEG	ARM: **CODE_BASE/out/arm_linux/debug/jpegdec** PC PCIE: **CODE_BASE/out/x86_linux_pci/debug/jpegdec**

From the command line, the desired TEST_BINARY can be run with parameters which are the same as those for the C-Model binary.

5 How to Verify VC9000D Hardware with C-Model

Before attempting verification, please make sure the following is in place:

- Assume the user has obtained the C-Model reference binary in Linux or Windows OS.
- `tb.cfg` should be added to the same directory with the C-Model reference binary before running the C-Model binary. `tb.cfg` can be found in the software release package.
- All drivers are installed. Both `memalloc` and the VC9000D device drivers are installed into the kernel successfully before running the FPGA binary

To verify FPGA against the C-Model, run the same parameter set with TEST_BINARY on FPGA and the C-Model binary.

C-model base compiling options:

```
make clean g2dec ENABLE_FPGA_VERIFICATION=y CLEAR_OUT_BUFFER=y USE_HW_PIC_DIMENSIONS=y
RELEASE=y
```

- For L2CACHE/Shaper, no extra options
- For DEC400, add option: SUPPORT_DEC400=y
- For VCMD, add option: USE_VCMD=y
- For MMU, no extra options
- For AXIFE, no extra options

FPGA base compiling options:

```
make clean g2dec ENV=x86_linux_pci  ENABLE_FPGA_VERIFICATION=y CLEAR_OUT_BUFFER=y
```

- For L2CACHE/Shaper, cache lib should be built in CACHE_BASE/software/linux_reference/ with command:
  ```
  make system
  ```
  Add option: SUPPORT_CACHE=y CACHE_DIR=CACHE_BASE in decoder compiling options.
  Add '-Axxx –pp-shaper' command line for PP0, the configuration of '-A' is related to specific project.
  Set `tb.cfg` for other PP units' configuration as follow (such as PP1):
  ```
  PpUnitParams {
    UnitIndex = 1;
    ShaperEnable = 1;
    ScaleWidth = 80;
    ScaleHeight = 68;
    UnitEnabled = 1;
  }
  ```
- For DEC400, add option: SUPPORT_DEC400=y
- For VCMD, add option: USE_VCMD=y
- For MMU, add option: SUPPORT_MMU=y
- For AXIFE, add option: SUPPORT_AXIFE=y

Use the following steps to compare the two decoded YUV/MD5 files, which should be bit-exact.

Step 1. Use the C-Model binary to run a series of cases. Each case will generate a corresponding YUV/MD5 file which can be used as "golden."

Step 2. Next, run the same cases with the same command line parameters using the test binary built for FPGA.

Step 3. If the decoded YUV/MD5 files of all cases generated by the FPGA are bit-exact with the ones generated by the C-Model, FPGA verification is passed.

Note: If DEC400 binding to L2CACHE/Shaper and there are differences with the verification, detailed steps and examples for 8-bit/10-bit stream verification are as follows (important options marked with **bold**):

Step 1. Use the C-Model binary to run a series of cases.

```
./g2dec -A256  -N5  -D156x144 -Ostream8bit.yuv stream8bit.hevc
./g2dec -A256  -N5  -D156x144 -Ep010 -Ostream10bit.yuv stream10bit.hevc.
```

Run the same cases using the test binary built for FPGA to get compressed data and table.

```
./g2dec -A256 --pp-shaper --dec400_enable -N5 -D156x144 -Ostream8bit_pp.yuv
stream8bit.hevc
./g2dec -A256 --pp-shaper --dec400_enable -N5 -D156x144 -Ep010 -
Ostream10bit_pp.yuv stream10bit.hevc
```

Step 2. Use VSIDecompress binary to decompress output YUV data to get decompressed data.

```
./VSIDecompress -i stream8bit_pp_0.yuv -bd 8bit -tm 256x1r -f NV12 -s 256 -w 156
-h 144 -a 32 -binIn -binOut  -o stream8bit_0.yuv -it
stream8bit_pp_pp_0_dec400_table.bin -n 5
./VSIDecompress -i stream8bit_pp_0.yuv -bd 10bit -tm 128x1r -f NV12 -s 256 -w
156 -h 144 -a 32 -binIn -binOut  -o stream10bit_0.yuv -it
stream10bit_pp_pp_0_dec400_table.bin -n 5
```

Step 3. If the output YUV data of all test cases from the C-Model are bit-exact as the ones generated by the decoder FPGA and `VSIDecompress` binary, FPGA verification is assumed to pass.

For the ARM platform, it is suggested to use UART control. Users can run the test binary and see the log via the UART console. A few recommended UART consoles for the Linux OS are `minicom` or `picocom`.

Internally, the VeriSilicon Hantro team has designed and used over 2000 cases to verify hardware support for all supported features as well as to exercise corner cases.

During a test, the input data needs to be loaded into the decoder's input buffers, and the decoder's output data needs to be read from the output buffers. Please confirm the data endian is correct in both the input buffer as well as the output buffer.

The decoder has software configurable registers to specify the data swap on BUS. For more detailed information, refer to the **Hantro VC9000D Video Decoder Hardware Integration Guide** document. The default BUS swap is set to no swap and the default endian is set to little-endian, which matches the endian in golden binary generated by the decoder C model.

It is recommended to use the default little-endian. If big endian must be used in the decoder's input and output buffer, the decoder's BUS swap configuration register must be changed accordingly to swap the endian.

To set the software register, refer to the source code file "`software\source\common\commonconfig.c`" which includes the following strings in the file:

- `HWIF_DEC_STRM_SWAP`
- `HWIF_DEC_PIC_SWAP`
- `HWIF_DEC_DIRMV_SWAP`
- `HWIF_DEC_TAB_SWAP`

6 How to Debug the Hardware Binary

If FPGA verification fails, it is necessary to debug TEST_BINARY to debug the issue. There are many debug methods:

1. Print register map
   User can open SOFTWARE_BASE/common/common.mk, enable macro "_DWL_DEBUG" and disable "DWL_DISABLE_REG_PRINTS" and build the hardware binary again.
   The register map will be printed if running a single case with TEST_BINARY.

2. Using **gdb**
   For the PC PCIE platform, gdb is easily installed. (For example: <apt-get install gdb>)
   For the ARM platform, the user should download the **gdb** package and build it with the cross compiler.
   Refer to the general **gdb** manual for details on how to build and use **gdb**.

7 Appendix A

This appendix describes the FPGA bring up using SOC tests. SOC tests can also be used to check the SoC environment and the FPGA synthesis of the VCCORE. These tests require a PC Linux OS or embedded Linux OS. The necessary data and buffers are prepared by ctrlSW and the OS, then ctrlSW sets the correct registers setting by kernel driver to VCCORE, then, the VCCORE can start one frame encoding or decoding.

7.1 Generating the Bit File

To verify RTL code on the FPGA, we need to map the RTL to the FPGA logic array. We introduce a reference flow and use “Synplify+Vivado” EDA tools.

7.1.1 Files Needed

- RTL
- Header files

7.1.2 FPGA Device-Related Files

- The DCM file is specific to the given Xilinx FPGA being used. It is essential to generate such a cell that is corresponding to the FPGA device. During generation of the DCM cell, make sure the frequency of this cell corresponds to the real clock frequency of the FPGA.
- FPGA cell library
- Synplify project file, constraints file
- Xilinx UCF (user constraints file)

7.1.3 Generating the Bit File

There are some things to remember when generating the bit file:

- Make sure the frequency definition of DCM matches the real clock frequency on the FPGA platform.
- Make sure the timing constraints are good enough, especially the I/O timing constraints you specify when using the ISE tool.
- Make sure the memories have been changed to FPGA memories.
  - Controlled by define: FPGA_SRAM; if you use Synplify tool to do the synthesize, add define: SYN_SRAM;
  - These defines will be used in files: vc9000d_vsram_dynamic.v, vc9000d_vdpsram_dynamic.v and vc9000d_vsrom_dynamic.v.
- Make sure the clock gating design has been removed.
  - Controlled by define : FPGA_CLKGATE;
  - This define will be used in file: vc9000d_clkgate.v

Bit file generation is performed in the following two steps:

1. Generate a netlist file using the Synplify tool.
   a. Create a project and choose the correct FPGA device.
   b. Add HDL define “FPGA_SRAM SYN_SRAM FPGA_CLKGATE” on this project.
   c. Read the RTL code into Synplify.
   d. Create timing constraints for this project.
   e. Run synthesize to produce a netlist file *.edf.
   f. Generate the bit file with the Vivado tool.
2. Create a project and choose the correct FPGA device.
   a. Read in *.edf file which is generated by Synplify.
   b. Set constraints, including those for timing and pin placement.

c. Run bit file generation to produce the *.bit file.

Document Revision History

This section describes top level differences in the versions of this document.
Note: This document is not necessarily updated for each patch or minor revision.

Document Revision	Date	Compatible Core	Description
1.02	2023-12-29	VC9000D series	• Updated the name of this document. • Updated the names of the reference documents.
1.01	2022-02-03	VC9000D	Legal Notices, General: update branding layout to include VeriSilicon.
1.00	2021-03-24	VC9000D	Initial